

Лабораторна робота №6.

Виявлення вразливості SQL-ін'єкцій

Мета: створити простий сценарій Python для виявлення вразливості SQL-ін'єкцій у веб-додатках.

Завдання:

1. Встановіть необхідні бібліотеки: *pip3 install requests bs4*.
2. Створіть новий файл `sql_injection_detector.py`, імпортуйте необхідні модулі, ініціалізуйте сеанс запитів і встановіть агент користувача:

```
1 import requests
2 from bs4 import BeautifulSoup as bs
3 from urllib.parse import urljoin
4 from pprint import pprint
5
6 s = requests.Session()
7 s.headers["User-Agent"] = "Chrome/83.0.4103.106"
```

3. Напишіть функцію `get_all_forms()` яка використовує бібліотеку **BeautifulSoup** для отримання всіх тегів форми з HTML і повертає їх як список Python:

```
10 def get_all_forms(url):
11     """Given a `url`, it returns all forms from the HTML content"""
12     soup = bs(s.get(url).content, "html.parser")
13     return soup.find_all("form")
```

4. Напишіть функцію `get_form_details()` яка отримує єдиний об'єкт тегу форми як аргумент і аналізує корисну інформацію про форму, таку як дія (цільова URL-адреса), метод (GET, POST тощо) і всі атрибути поля введення (тип, ім'я та значення).

```
16 def get_form_details(form):
17     """
18     This function extracts all possible useful information about an HTML `form`
19     """
20     details = {}
21     # get the form action (target url)
22     try:
23         action = form.attrs.get("action").lower()
24     except:
25         action = None
26     # get the form method (POST, GET, etc.)
27     method = form.attrs.get("method", "get").lower()
28     # get all the input details such as type and name
29     inputs = []
30     for input_tag in form.find_all("input"):
31         input_type = input_tag.attrs.get("type", "text")
32         input_name = input_tag.attrs.get("name")
33         input_value = input_tag.attrs.get("value", "")
34         inputs.append({"type": input_type, "name": input_name, "value": input_value})
35     # put everything to the resulting dictionary
36     details["action"] = action
37     details["method"] = method
38     details["inputs"] = inputs
39     return details
```

5. Визначаємо функцію, яка повідомляє нам, чи є на веб-сторінці помилки SQL, це буде зручно під час перевірки вразливості SQL-ін'єкції:

```

42 def is_vulnerable(response):
43     """A simple boolean function that determines whether a page
44     is SQL Injection vulnerable from its `response`"""
45     errors = {
46         # MySQL
47         "you have an error in your sql syntax;",
48         "warning: mysql",
49         # SQL Server
50         "unclosed quotation mark after the character string",
51         # Oracle
52         "quoted string not properly terminated",
53     }
54     for error in errors:
55         # if you find one of these errors, return True
56         if error in response.content.decode().lower():
57             return True
58     # no error detected
59     return False

```

6. Визначаємо основну функцію, яка шукає всі форми на веб-сторінці та намагається розмістити символи лапок і подвійних лапок у полях введення:

```

62 def scan_sql_injection(url):
63     # test on URL
64     for c in "\"'":
65         # add quote/double quote character to the URL
66         new_url = f"{url}{c}"
67         print("[!] Trying", new_url)
68         # make the HTTP request
69         res = s.get(new_url)
70         if is_vulnerable(res):
71             # SQL Injection detected on the URL itself,
72             # no need to proceed for extracting forms and submitting them
73             print("[+] SQL Injection vulnerability detected, link:", new_url)
74             return
75     # test on HTML forms
76     forms = get_all_forms(url)
77     print(f"[+] Detected {len(forms)} forms on {url}.")
78
79     for form in forms:
80         form_details = get_form_details(form)
81         for c in "\"'":
82             # the data body we want to submit
83             data = {}
84             for input_tag in form_details["inputs"]:
85                 if input_tag["value"] or input_tag["type"] == "hidden":
86                     # any input form that has some value or hidden,
87                     # just use it in the form body
88                     try:
89                         data[input_tag["name"]] = input_tag["value"] + c
90                     except:
91                         pass
92                 elif input_tag["type"] != "submit":
93                     # all others except submit, use some junk data with special character
94                     data[input_tag["name"]] = f"test{c}"
95             # join the url with the action (form request URL)
96             url = urljoin(url, form_details["action"])
97             if form_details["method"] == "post":
98                 res = s.post(url, data=data)
99             elif form_details["method"] == "get":
100                 res = s.get(url, params=data)
101             # test whether the resulting page is vulnerable
102             if is_vulnerable(res):
103                 print("[+] SQL Injection vulnerability detected, link:", url)
104                 print("[+] Form:")
105                 pprint(form_details)
106                 break

```

Перш ніж видобувати форми та надсилати їх, наведена вище функція спочатку перевіряє URL-адресу на наявність уразливості, оскільки сама URL-адреса може бути вразливою. Це робиться просто

додаванням символу лапки до URL-адреси. Потім ми робимо запит за допомогою бібліотеки запитів і перевіряємо, чи вміст відповіді містить помилки, які ми шукаємо. Після цього ми аналізуємо форми та робимо надсилення із символами лапок у кожній знайдений формі.

7. Викликаємо основну функцію:

```
107 if __name__ == "__main__":  
108     import sys  
109     url = sys.argv[1]  
110     scan_sql_injection(url)
```

8. Запускаємо скрипт командою:

python3 sql_injection_detector.py *http://testphp.vulnweb.com/artists.php?artist=1*

9. Спробуйте виявити вразливість SQL-ін'єкцій у інших веб-додатках. Запишіть результати роботи **sql_injection_detector.py**.